



AUDIT REPORT

June 2026

For



Table of Content

Executive Summary	04
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	07
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
 High Severity Issues	14
1. Storage Layout Collision Breaks On-Chain Message Signing and Can Corrupt Recovery State	14
2. Recovery Execution Is An Unconstrained Arbitrary-Call Lane That Ignores jrc.to	15
 Medium Severity Issues	16
3. Recovery Event Stream Is Spoofable And Cannot Serve As Authoritative Proof	16
Functional Tests	17
Threat Model	20
Automated Tests	50
Closing Summary & Disclaimer	51



Executive Summary

Project Name	Jurat
Protocol Type	{Safe} Wallet fork
Project URL	https://www.jurat.io/
Overview	The reviewed system forks the Safe wallet architecture and adds Jurat-specific recovery, a public recovery event emitter, a fee-enabled proxy factory, and a fee-charging multisend variant.
Audit Scope	The scope of this audit was to analyze the Jurat Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/jurat-systems/jurat-smart-wallets
Branch	main
Contracts in Scope	contracts/JuratSafe.sol contracts/JuratSafeL2.sol contracts/RecoveryEventEmitter.sol contracts/base/RecoveryManager.sol contracts/libraries/MultiSendFees.sol contracts/structs/JuratRecoveryCode.sol contracts/proxies/SafeProxyFactoryWithFees.sol contracts/interfaces/ISafeFactory.sol contracts/interfaces/IRecoveryEmitter.sol
Commit Hash	850e0ec
Language	Solidity
Blockchain	EVM
Method	Manual Analysis, Threat Modelling
Review 1	15th - 22nd June, 2026
Updated Code Received	25th June, 2026
Review 2	25th - 26th June, 2026
Fixed In	9b8f72e

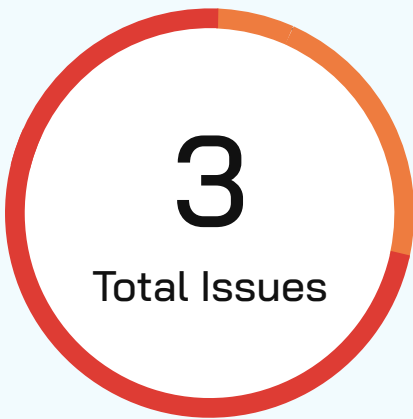


Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0%)
High	2 (66.67%)
Medium	1 (33.33%)
Low	0 (0%)
Informational	0 (0%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	1	0	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	2	0	0	0



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Storage Layout Collision Breaks On-Chain Message Signing and Can Corrupt Recovery State	High	Resolved
2	Recovery Execution Is An Unconstrained Arbitrary-Call Lane That Ignores jrc.to	High	Resolved
3	Recovery Event Stream Is Spoofable And Cannot Serve As Authoritative Proof	Medium	Acknowledged



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Manual analysis, threat modelling, functional testing.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



High Severity Issues

Storage Layout Collision Breaks On-Chain Message Signing and Can Corrupt Recovery State

Resolved

Path

RecoveryManager.sol, JuratSafe.sol

Function Name

JuratSafe storage layout vs SafeStorage, SignMessageLib.signMessage()

Description

RecoveryManager (a parent of JuratSafe) declares proxyFactory and recoveryEmitter, which take storage slots 5 and 6, ahead of JuratSafe's own variables, since parent storage is always laid out before the child's. This shifts signedMessages from slot 7 (where the canonical SafeStorage layout used by all Safe libraries places it) to slot 9:

REAL JuratSafe	SafeStorage blueprint
slot 5 proxyFactory (inserted)	slot 5 nonce
slot 6 recoveryEmitter (inserted)	slot 6 _deprecatedDomainSeparator
slot 7 nonce	slot 7 signedMessages
slot 8 _deprecatedDomainSeparator	slot 8 approvedHashes
slot 9 signedMessages	

When a wallet delegatecalls SignMessageLib to sign a message, the library writes the approval to the slot-7 location while the wallet reads it from slot 9, and the two never meet.

Impact

On-chain EIP-1271 message signing is silently broken for every wallet: signMessage succeeds but isValidSignature later reverts "Hash not approved". The same misalignment also lets any SafeStorage-based library that writes a scalar at slot 5 or 6 overwrite proxyFactory or recoveryEmitter, disabling or redirecting recovery authority.

Recommendation

Store the recovery variables at fixed hash-derived slots (the pattern Safe uses for the fallback handler and guard) so they leave the sequential range and signedMessages returns to slot 7. Reordering the inheritance list does not work, since parent variables always precede the child's.

```
// keccak256("jurat.recovery_manager.proxy_factory")
bytes32 internal constant PROXY_FACTORY_SLOT =
0x2c0172a6775543e3d8a148f1667180b224e5062f9775128dd6c3854baa7a46c7;

// keccak256("jurat.recovery_manager.recovery_emitter")
bytes32 internal constant RECOVERY_EMITTER_SLOT =
0xbc27f3a45e0c04b07b85770e902d1f85ae0adf2bc127f230f116931a56609fb7;

// setupRecovery sstores into these slots; getProxyFactory()/
getRecoveryEmitter() sload them.
```



High Severity Issues

Recovery Execution Is An Unconstrained Arbitrary-Call Lane That Ignores jrc.to

Resolved

Path

RecoveryManager.sol

Function Name

`executeRecovery()`

Description

`executeRecovery` validates `from`, `cid`, `chainId`, and a non-empty `callInfo` array, then loops over `callInfo` and runs raw calls from the Safe:

```
for (uint256 i = 0; i < infoLength; i++) {
    _call(jrc.callInfo[i].dest, jrc.callInfo[i].value,
jrc.callInfo[i].data);
}
```

It never reads or enforces `JuratRecoveryCode.to`. Every destination, value, selector, and token effect comes from `callInfo`, so `to address` is a decorative label with no on-chain effect. Recovery is therefore arbitrary admin execution, not a constrained sweep to the displayed beneficiary.

Impact

The admin can move any asset, in any amount, to any address, independent of the `jrc.to` carried in the recovery code and echoed in the `RecoveryExecuted` event. Off-chain reviewers or approval tooling that inspect only `jrc.to` can sign off on recoveries that actually send funds elsewhere, and the emitted event misrepresents the true recipient.

Recommendation

Either constrain recovery to asset sweeps whose recipient parameters must equal `jrc.to`.



Medium Severity Issues

Recovery Event Stream Is Spoofable And Cannot Serve As Authoritative Proof

Acknowledged

Path

RecoveryEventEmitter.sol

Function Name

`RecoveryEventEmitter.executedRecovery()`

Description

RecoveryManager emits recovery completion through an external RecoveryEventEmitter, but `executedRecovery` is public and unauthenticated:

```
function executedRecovery(JuratRecoveryCode calldata jrc) public {  
    emit RecoveryExecuted(jrc);  
}
```

Anyone can call it directly with an arbitrary JuratRecoveryCode and emit a RecoveryExecuted event that never corresponded to a real recovery.

Impact

Off-chain indexers, dashboards, or monitoring that treat the RecoveryExecuted stream as proof of a recovery can be fed fabricated events with attacker-chosen from, to, cid, and reason. The event stream cannot be used as authoritative evidence of a recovery without correlating it against the actual on-chain `executeRecovery` transaction.

Recommendation

Emit recovery events directly from the Safe recovery logic, or restrict the emitter to authenticated Safe or recovery-manager callers.

Jurat Team's Comment :

We intend to make use of the recovery event emitter contract as a way to centrally collect data on the various recovery actions executed across the deployed smart wallet instances, for statistical purposes. Centralizing this on-chain query makes it simpler than monitoring all Safe instances that may trigger these actions and this event in particular over time. Although it may be the case that any external party could trigger the `executedRecovery()` method, we intend to collect and filter all of this data off-chain, so that no calls to the contract external to our organization could affect the outcome of that statistical analysis.



Functional Tests

JuratSafe :

- ✓ setup can only run once (setupOwners requires threshold is currently zero)
- ✓ setup reverts if the threshold is zero
- ✓ setup reverts if the threshold is greater than the number of owners
- ✓ setup reverts on a zero, sentinel, self, or duplicate owner address
- ✓ setup sets the fallback handler only when a non-zero handler is provided
- ✓ setup runs the optional module-setup delegatecall only when `to` is non-zero
- ✓ setup wires the proxy factory and recovery emitter via setupRecovery
- ✓ setup reverts if the recovery emitter address is zero
- ✓ setup stores the deploying factory as the `factory` state variable
- ✓ setup charges the optional one-time deployment payment only when payment is above zero
- ✓ setup emits SafeSetup with the initiator, owners, threshold, and handler
- ✓ execTransaction reverts when the provided signatures do not meet the threshold
- ✓ execTransaction increments the nonce before executing the inner call
- ✓ execTransaction runs the guard pre-check when a guard is configured
- ✓ execTransaction runs the guard post-check when a guard is configured
- ✓ execTransaction reverts when not enough gas remains for safeTxGas
- ✓ execTransaction requires inner success when both safeTxGas and gasPrice are zero
- ✓ execTransaction pays the gas refund to the receiver only when gasPrice is above zero
- ✓ execTransaction emits ExecutionSuccess when the inner call succeeds
- ✓ execTransaction emits ExecutionFailure when the inner call fails
- ✓ execTransaction sends the protocol fee to the factory owner when the fee is non-zero
- ✓ handlePayment caps the ETH refund at the actual transaction gas price



- ✓ `handlePayment` pays the refund in the gas token when one is specified
- ✓ `checkSignatures` reverts when the threshold is zero
- ✓ `checkNSignatures` reverts when the signature blob is shorter than required
- ✓ `checkNSignatures` recovers EOA signatures with `ecrecover`
- ✓ `checkNSignatures` supports `eth_sign` signatures when `v` is above 30
- ✓ `checkNSignatures` supports pre-approved hash signatures when `v` is one
- ✓ `checkNSignatures` supports EIP-1271 contract signatures when `v` is zero
- ✓ `checkNSignatures` enforces strictly ascending owner order to block duplicates
- ✓ `checkNSignatures` rejects any signer that is not a registered owner
- ✓ `approveHash` reverts when the caller is not an owner
- ✓ `approveHash` records the hash as approved for the calling owner
- ✓ `getChainId` returns the current chain id
- ✓ `domainSeparator` returns the EIP-712 domain separator for this contract
- ✓ `getTransactionHash` returns the EIP-712 hash that owners must sign

RecoveryManager :

- ✓ `setupRecovery` reverts if the factory address is zero
- ✓ `setupRecovery` reverts if the emitter address is zero
- ✓ `setupRecovery` stores the proxy factory and recovery emitter
- ✓ `onlyAdmin` allows only the current factory owner to proceed
- ✓ `executeRecovery` reverts when the caller is not the admin
- ✓ `executeRecovery` reverts when `jrc.from` is not this wallet
- ✓ `executeRecovery` reverts when the cid string is empty
- ✓ `executeRecovery` reverts when `jrc.chainId` does not match the current chain
- ✓ `executeRecovery` reverts when the `callInfo` array is empty
- ✓ `executeRecovery` runs each `callInfo` entry as a call from the wallet



- ✓ `executeRecovery` forwards the value and data of each `callInfo` entry
- ✓ `executeRecovery` bubbles up the revert reason of any failed inner call
- ✓ `executeRecovery` emits `RecoveryExecuted` through the recovery emitter
- ✓ `getProxyFactory` returns the stored proxy factory
- ✓ `getRecoveryEmitter` returns the stored recovery emitter
- ✓ `proxyFactoryAdmin` returns the owner of the proxy factory
- ✓ `executeRecovery` validates `jrc.to`

SafeProxyFactoryWithFees :

- ✓ constructor sets the admin as the owner and stores the initial fee
- ✓ constructor emits `FeeSet` with the initial fee
- ✓ `createProxyWithNonce` deploys a proxy and runs the initializer call
- ✓ `createProxyWithNonce` reverts when the singleton address has no code
- ✓ `createProxyWithNonce` reverts when the `CREATE2` deployment returns the zero address
- ✓ `createProxyWithNonce` reverts when the initializer call fails
- ✓ `createProxyWithNonce` emits `ProxyCreation` with the proxy and singleton
- ✓ `createChainSpecificProxyWithNonce` folds the chain id into the `CREATE2` salt
- ✓ `createProxyWithCallback` invokes the creation callback when one is provided
- ✓ `setFee` reverts when the caller is not the owner
- ✓ `setFee` updates the fee and emits `FeeSet`
- ✓ ownership transfer is two-step via `transferOwnership` then `acceptOwnership`
- ✓ `getChainId` returns the current chain id
- ✓ `proxyCreationCode` returns the proxy creation bytecode



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Factory owner / Jurat governance	Admin / identity	Returned by <code>ISafeFactory.owner()</code> and used as recovery admin, fee recipient, and <code>setFee</code> authority.	The owner key/process remains honest, available, and correctly rotated for the lifetime of deployed wallets.	Compromise or unsafe ownership changes can affect recovery execution, fee configuration, and liveness for already-created wallets.
Factory fee() value	Mutable protocol configuration	Read by <code>JuratSafe.execTransaction</code> and <code>MultiSendFees.multiSend</code> to determine the flat ETH fee.	Fee remains reasonable, payable, and intentionally updated.	Token-only wallets can be unable to transact; users do not sign the post-execution Jurat fee terms.
User-supplied singleton and initializer	Deployment trust boundary	Public factory functions accept <code>_singleton</code> and initializer and deploy proxies that run the initializer.	Users, scripts, and integrators verify that singleton and initializer target the intended Jurat implementation and setup payload.	Incorrect singleton or initializer can permanently bind wrong owners, modules, fallback handler, factory, recovery emitter, or fee behavior.



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Callback contract in <code>createProxyWithCallback</code>	External deployment hook	Invoked after proxy deployment and initialization if a nonzero callback is supplied.	Callback is intentional and safe to call during deployment.	Callback reverts can revert deployment; malicious callbacks can emit misleading metadata or perform unexpected external effects.
Recovery event consumers	Offchain / indexing	Observe <code>RecoveryExecuted(JuratRecoveryCode)</code> logs emitted by <code>RecoveryEventEmitter</code> .	Consumers validate transaction sender/context and do not trust logs by event signature alone.	Public emitter can be spoofed; event-only systems can mistake forged recovery logs for authoritative executions.
Enabled Safe modules, including ERC-4337 module flows	Onchain module / account abstraction	Enabled modules can call <code>execTransactionFromModule</code> without owner signatures.	Only audited and intentionally enabled modules are installed.	Module compromise or misconfiguration can move assets, install additional modules through self-calls, or bypass owner threshold checks.
EntryPoint / bundler infrastructure	Runtime / account abstraction	ERC-4337 module interactions depend on EntryPoint-style execution and simulation assumptions.	EntryPoint address and bundler behavior are correct for the target chain.	Incorrect EntryPoint or bundler assumptions can route module execution unexpectedly or affect liveness.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Fallback handler	Onchain extension	Handles unknown calls forwarded by the Safe fallback path and may append caller context.	Handler is correct, intentionally installed, and compatible with Jurat/Safe assumptions.	Selector routing or handler bugs can expose unintended logic or confuse offchain decoding.
Guards	Onchain policy hook	Pre- and post-check Safe owner-signed transactions.	Guard code is correct and removable under safe conditions.	Broken or malicious guards can freeze valid owner-signed transactions.
Arbitrary call targets	External contracts / users	execTransaction, module execution, recovery, and multisend can call arbitrary targets.	Calldata, target behavior, and return semantics are understood before execution.	Malicious targets, nonstandard return values, reentrancy, or calldata mismatch can move assets contrary to displayed intent.
Token contracts	Asset contracts	Wallets can hold and transfer ERC20/ERC721/ERC1155 or nonstandard tokens through arbitrary calls.	Token behavior is standard enough for signing, simulation, recovery, and monitoring tools.	Fee-on-transfer, rebasing, false-return, blacklisting, or callback tokens can break accounting or recovery assumptions.



2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
JuratSafe / JuratSafeL2	receive()	What this function can do Accept native currency into the Safe proxy and emit SafeReceived. What this function cannot/should not do It should not authenticate sender intent or update owner/module/recovery state. Main invariant(s) Native funds received become wallet assets controlled by Safe execution paths. Access level Public payable.
JuratSafe / JuratSafeL2	fallback()	What this function can do Forward calldata to the configured fallback handler with caller context appended. What this function cannot/should not do It should not be treated as owner approval or recovery authorization. Main invariant(s) Only the configured fallback handler receives forwarded unknown selectors; handler must not be the Safe itself. Access level Public



Contract Name	Function	Category
JuratSafe / JuratSafeL2	setup(address s[],uint256,ad dress,bytes,a ddress,addres s,uint256,add ress,address)	What this function can do Initialize owners, threshold, modules, fallback handler, payment, recovery emitter, and stored factory binding. What this function cannot/should not do It should not be callable more than once or with hidden unreviewed initializer effects. Main invariant(s) Threshold is set once; setup permanently defines core wallet control-plane bindings. Access level Permissionless before initialization; intended to be called atomically by the factory.
JuratSafe	execTransacti on(address,ui nt256,bytes,E num.Operatio n,uint256,uint 256,uint256,a ddress,addres s,bytes)	What this function can do Execute arbitrary call or delegatecall after validating owner signatures, then optionally pay Safe gas refund and Jurat ETH fee. What this function cannot/should not do It should not execute without threshold authorization, and fee terms should not exceed user-visible expectations. Main invariant(s) Nonce advances before signature validation; successful authorization gates arbitrary wallet execution; post-execution fee can revert the full transaction. Access level Public payable with valid Safe signatures or approved hashes.

Contract Name	Function	Category
JuratSafeL2	execTransaction(...)	What this function can do Emit richer transaction metadata and then delegate to JuratSafe.execTransaction semantics. What this function cannot/should not do It should not diverge from base execution authorization or fee behavior. Main invariant(s) L2 event metadata should match the executed transaction parameters. Access level Public payable with valid Safe signatures or approved hashes.
JuratSafe / JuratSafeL2	approveHash(bytes32)	What this function can do Mark a hash as approved by an owner for future signature checks. What this function cannot/should not do It should not allow non-owners to approve hashes or revoke prior approvals. Main invariant(s) Only current owners can set approvedHashes[msg.sender][hash]. Access level Owner-only by owners[msg.sender] != address(0).

Contract Name	Function	Category
JuratSafe / JuratSafeL2	checkSignatures(bytes32,bytes,bytes)	What this function can do Validate signatures against the current threshold. What this function cannot/should not do It should not mutate wallet state or lower required signatures. Main invariant(s) Signature set must satisfy threshold and owner ordering constraints. Access level Public view.
JuratSafe / JuratSafeL2	checkNSignatures(bytes32,bytes,bytes,uint256)	What this function can do Validate an explicit number of signatures for a data hash. What this function cannot/should not do It should not authorize non-owner signatures or out-of-bounds contract-signature payloads. Main invariant(s) Each counted signer must be a unique enabled owner and satisfy ECDSA, approved-hash, or EIP-1271 validation. Access level Public view.

Contract Name	Function	Category
JuratSafe / JuratSafeL2	enableModule (address)	What this function can do Add a module to the enabled-module linked list. What this function cannot/should not do It should not be directly callable by arbitrary external accounts. Main invariant(s) Module is nonzero, not sentinel, and not already enabled. Access level Self-authorized Safe call.
JuratSafe / JuratSafeL2	disableModule(address, address)	What this function can do Remove an enabled module from the linked list. What this function cannot/should not do It should not remove unrelated modules or accept incorrect previous pointers. Main invariant(s) modules[prevModule] == module must hold. Access level Self-authorized Safe call.
JuratSafe	executeTransactionFromModule(address, uint256, bytes, Enum.Operation)	What this function can do Let an enabled module execute arbitrary call or delegatecall from the Safe without owner signatures. What this function cannot/should not do It should not allow disabled modules or sentinel values to execute.



Contract Name	Function	Category
JuratSafeL2	execTransactionFromModule(...)	Main invariant(s) modules[msg.sender] != address(0) gates module execution. Access level Enabled module only. What this function can do Emit SafeModuleTransaction and then execute through the base module path. What this function cannot/should not do It should not weaken module authorization or omit event data needed by indexers. Main invariant(s) Event fields should match the module transaction actually attempted. Access level Enabled module only.
JuratSafe / JuratSafeL2	execTransactionFromModuleReturnData(address,uint256,bytes,Enum.Operation)	What this function can do Execute a module transaction and return the low-level returndata. What this function cannot/should not do It should not bypass enabled-module checks. Main invariant(s) Return data corresponds to the module execution result. Access level Enabled module only.

Contract Name	Function	Category
JuratSafe / JuratSafeL2	addOwnerWithThreshold(address,uint256)	What this function can do Add a new owner and optionally change the threshold. What this function cannot/should not do It should not be callable by an external account except through Safe self-execution. Main invariant(s) Owner is nonzero, not sentinel, not the Safe itself, and not already present. Access level Self-authorized Safe call.
JuratSafe / JuratSafeL2	removeOwner(address,uint256)	What this function can do Remove an owner and optionally change threshold. What this function cannot/should not do It should not remove owners with an invalid linked-list predecessor or leave threshold unreachable. Main invariant(s) owners[prevOwner] == owner and ownerCount - 1 >= newThreshold must hold. Access level Self-authorized Safe call.

Contract Name	Function	Category
JuratSafe / JuratSafeL2	swapOwner(address,address)	What this function can do Replace one owner with another. What this function cannot/should not do It should not add duplicate, zero, sentinel, or Safe-self owners. Main invariant(s) Linked-list predecessor must point to the old owner; new owner must be unused. Access level Self-authorized Safe call.
JuratSafe / JuratSafeL2	changeThreshold(uint256)	What this function can do Change required owner confirmations. What this function cannot/should not do It should not set threshold to zero or above owner count. Main invariant(s) $1 \leq \text{threshold} \leq \text{ownerCount}$. Access level Self-authorized Safe call.
JuratSafe / JuratSafeL2	setFallbackHandler(address)	What this function can do Set or clear fallback handler routing. What this function cannot/should not do It should not set the Safe itself as fallback handler. Main invariant(s) Fallback handler storage slot reflects intentional owner-authorized configuration. Access level Self-authorized Safe call.



Contract Name	Function	Category
JuratSafe / JuratSafeL2	setGuard(address)	What this function can do Set or clear the transaction guard. What this function cannot/should not do It should not set a non-guard contract as guard. Main invariant(s) Nonzero guard must support the Safe guard interface. Access level Self-authorized Safe call.
RecoveryManager / JuratSafe	executeRecovery(JuratRecoveryCode)	What this function can do Let the current factory owner execute each callInfo item from the Safe and emit recovery metadata through the emitter. What this function cannot/should not do It should not be treated as owner-signed Safe execution, and should not assume jrc.to controls actual recipients. Main invariant(s) jrc.from == address(this), nonempty cid, matching chainId, and nonempty callInfo; current implementation does not constrain targets, selectors, or recipients. Access level Current factory owner returned by proxyFactoryAdmin().



Contract Name	Function	Category
JuratSafe / JuratSafeL2	simulateAndRevert(address,bytes)	What this function can do Delegatecall a target for simulation and revert with encoded success/response data. What this function cannot/should not do It should not persist side effects because it intentionally reverts. Main invariant(s) Caller must treat revert data as simulation output, not failure alone. Access level Public
JuratSafe / JuratSafeL2	getStorageAt(uint256,uint256)	What this function can do Expose arbitrary storage words for indexing and inspection. What this function cannot/should not do It should not mutate storage or hide privileged configuration. Main invariant(s) Returned data mirrors raw storage at requested offsets. Access level Public view.
JuratSafe / JuratSafeL2	getThreshold()	What this function can do Expose the current owner-signature threshold What this function cannot/should not do It should not change threshold. Main invariant(s) Threshold remains within owner-count bounds after valid mutations. Access level Public view.



Contract Name	Function	Category
JuratSafe / JuratSafeL2	isOwner(addr ess)	What this function can do Expose whether an address is a current owner What this function cannot/should not do It should not add or remove owners. Main invariant(s) Owner membership reflects the owner linked list. Access level Public view.
JuratSafe / JuratSafeL2	getOwners()	What this function can do Return the current owner set. What this function cannot/should not do It should not reorder or mutate owners. Main invariant(s) Returned owners should match the linked-list state and owner count. Access level Public view.
JuratSafe / JuratSafeL2	isModuleEnabled(address)	What this function can do Expose whether a module is enabled. What this function cannot/should not do It should not authorize or execute modules by itself. Main invariant(s) Enabled modules are non-sentinel and present in module mapping. Access level Public view.

Contract Name	Function	Category
JuratSafe / JuratSafeL2	getModulesPaginated(address,uint256)	What this function can do Return paginated enabled-module list. What this function cannot/should not do It should not mutate module state. Main invariant(s) Pagination start must be sentinel or enabled module; page size must be positive. Access level External view.
RecoveryManager / JuratSafe	getRecoveryEmitter()	What this function can do Expose the configured recovery emitter address. What this function cannot/should not do It should not authenticate emitted logs by itself. Main invariant(s) Returned emitter is the address set during setup. Access level Public view.

Contract Name	Function	Category
RecoveryManager / JuratSafe	getModulesP aginated(addr ess,uint256)	What this function can do Expose the stored factory address used for recovery admin lookup. What this function cannot/should not do It should not prove the factory owner is safe or immutable. Main invariant(s) Returned factory is the address set during setup. Access level Public view.
RecoveryManager / JuratSafe	proxyFactory Admin()	What this function can do Resolve the current factory owner used by onlyAdmin. What this function cannot/should not do It should not snapshot recovery authority. Main invariant(s) Recovery authorization tracks live ISafeFactory(proxyFactory).owner(). Access level Public view.

Contract Name	Function	Category
JuratSafe / JuratSafeL2	factory()	What this function can do Expose the factory address used by direct fee logic. What this function cannot/should not do It should not prove fee terms are immutable or signer-approved. Main invariant(s) Returned factory was stored during setup and is queried for live fee() and owner() values. Access level Public view autogenerated getter.
JuratSafe / JuratSafeL2	nonce()	What this function can do Expose the next Safe transaction nonce used in owner-signed transaction hashes. What this function cannot/should not do It should not be manually decremented or reused. Main invariant(s) Each successful execTransaction attempt consumes the current nonce before signature validation completes. Access level Public view autogenerated getter.

Contract Name	Function	Category
JuratSafe / JuratSafeL2	approvedHashes(address,bytes32)	What this function can do Expose whether an owner pre-approved a hash. What this function cannot/should not do It should not imply the approving address is still an owner without checking current owner state. Main invariant(s) Approved hashes persist once set and are consumed by signature validation rules. Access level Public view autogenerated getter.
JuratSafe / JuratSafeL2	signedMessages(bytes32)	What this function can do Expose hashes marked as signed messages by the Safe message-signing flow. What this function cannot/should not do It should not move assets or authorize Safe transactions by itself. Main invariant(s) Message-signing state must remain separate from transaction execution authorization. Access level Public view autogenerated getter.

Contract Name	Function	Category
JuratSafe / JuratSafeL2	getChainId()	What this function can do Return current EVM chain ID used by Safe domain separation. What this function cannot/should not do It should not detect wrong RPC or deployment environment by itself. Main invariant(s) Value equals EVM chainid(). Access level Public view.
JuratSafe / JuratSafeL2	domainSeparator()	What this function can do Return the EIP-712 domain separator for the Safe and chain. What this function cannot/should not do It should not include Jurat fee or recovery terms. Main invariant(s) Domain binds signatures to chainId and verifying Safe address. Access level Public view.

Contract Name	Function	Category
JuratSafe / JuratSafeL2	encodeTransactionData(...)	What this function can do Return the EIP-712 Safe transaction preimage for owner signing. What this function cannot/should not do It should not be assumed to cover Jurat recovery calls or post-execution flat fees. Main invariant(s) Encoded data must match SAFE_TX_TYPEHASH fields and Safe nonce supplied. Access level Public view.
JuratSafe / JuratSafeL2	getTransactionHash(...)	What this function can do Return the Safe transaction hash owners sign or approve. What this function cannot/should not do It should not imply execution success or fee consent beyond encoded fields. Main invariant(s) Hash equals keccak256(encodeTransactionData(...)). Access level Public view.

Contract Name	Function	Category
RecoveryEventEmitter	executedRecovery(JuratRecoveryCode)	What this function can do Emit RecoveryExecuted(jrc) for any supplied recovery struct. What this function cannot/should not do It should not be treated as proof that RecoveryManager.executeRecovery ran. Main invariant(s) No authentication is enforced by the emitter. Access level Public.
SafeProxyFactoryWithFees	createProxyWithNonce(address,bytes,uint256)	What this function can do Deploy a Safe proxy with CREATE2 and optionally execute initializer. What this function cannot/should not do It should not silently hide initializer/factory/recovery bindings from users. Main invariant(s) Proxy address is determined by singleton, initializer hash, and salt nonce. Access level Public.

Contract Name	Function	Category
SafeProxyFactoryWithFees	createChainSpecificProxyWithNonce(address,bytes,uint256)	What this function can do Deploy a proxy with chain ID included in the CREATE2 salt. What this function cannot/should not do It should not be assumed replayable across chains. Main invariant(s) Proxy address derives from initializer hash, salt nonce, and chainid(). Access level Public.
SafeProxyFactoryWithFees	createProxyWithCallback(address,bytes,uint256,IProxyCreationCallback)	What this function can do Deploy and initialize a proxy, then invoke a callback with deployment metadata. What this function cannot/should not do It should not treat arbitrary callbacks as trusted security controls. Main invariant(s) Callback-specific salt affects proxy address; callback revert reverts the deployment transaction. Access level Public.

Contract Name	Function	Category
SafeProxyFactoryWithFees	setFee(uint256)	What this function can do Update the flat ETH fee charged by Jurat execution paths that read the factory. What this function cannot/should not do It should not bypass governance controls, caps, or user-visible fee assumptions. Main invariant(s) fee reflects the latest owner-set value. Access level Factory owner only.
SafeProxyFactoryWithFees	transferOwnership(address)	What this function can do Start transferring factory ownership to a pending owner through Ownable2Step. What this function cannot/should not do It should not instantly transfer recovery/fee authority without acceptance. Main invariant(s) Pending owner must explicitly accept before becoming owner. Access level Current factory owner.
SafeProxyFactoryWithFees	acceptOwnership()	What this function can do Finalize two-step ownership transfer. What this function cannot/should not do It should not be callable by anyone except the pending owner. Main invariant(s) Factory owner after acceptance becomes the live recovery admin and fee recipient for wallets resolving factory owner dynamically.



Contract Name	Function	Category
SafeProxyFactoryWithFees	renounceOwnership()	Access level Pending owner. What this function can do Remove factory ownership if inherited Ownable behavior is reachable. What this function cannot/should not do It should not be used unless loss of recovery admin and fee ownership is intended. Main invariant(s) No-owner state can break owner-gated operations that require a nonzero admin.
SafeProxyFactoryWithFees	fee()	Access level Current factory owner. What this function can do Expose the current flat ETH fee. What this function cannot/should not do It should not prove that a user signed or accepted the fee. Main invariant(s) Execution paths reading fee() use the current value, not a wallet snapshot. Access level Public view.

Contract Name	Function	Category
SafeProxyFactoryWithFees	owner()	What this function can do Expose current factory owner. What this function cannot/should not do It should not guarantee key safety or governance process quality. Main invariant(s) Returned owner is the live admin for setFee and recovery authorization lookups. Access level Public view.
SafeProxyFactoryWithFees	pendingOwner()	What this function can do Expose the pending owner during two-step ownership transfer. What this function cannot/should not do It should not grant authority until acceptOwnership() succeeds. Main invariant(s) Pending owner is the only account that can finalize ownership transfer. Access level Public view inherited getter.
SafeProxyFactoryWithFees	proxyCreationCode()	What this function can do Expose proxy creation bytecode for address prediction. What this function cannot/should not do It should not validate singleton or initializer correctness.

Contract Name	Function	Category
SafeProxyFactoryWithFees	getChainId()	Main invariant(s) Returned bytecode must match deployed proxy creation code. Access level Public pure. What this function can do Return current chain ID used by chain-specific deployment salts. What this function cannot/should not do It should not detect wrong RPC or fork context by itself. Main invariant(s) Value equals EVM chainid(). Access level Public view.
MultiSendFees	multiSend(bytes)	What this function can do Execute a packed batch of calls/delegatecalls in caller storage context when invoked through delegatecall, then pay the factory owner fee. What this function cannot/should not do It should not be called directly as the singleton implementation; it should not hide batch side effects from signers. Main invariant(s) address(this) != multisendSingleton must hold, so intended use is delegatecall through a Safe transaction. Access level Public payable, but effectively delegatecall-only.



3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
JuratSafe proxy instances	Native currency, ERC20, ERC721, ERC1155, and arbitrary tokenized assets	User wallet balances held at each Safe proxy address.
JuratSafeL2 proxy instances	Native currency, ERC20, ERC721, ERC1155, and arbitrary tokenized assets	Same wallet custody model as JuratSafe, with richer transaction/module events.
SafeProxyFactoryWithFees	None in reviewed contracts	Factory deploys proxies and stores fee configuration; no normal asset custody path was identified in scoped code.
RecoveryEventEmitter	None in reviewed contracts	Emits recovery metadata only; no payable or token custody function was identified.
MultiSendFees	None in reviewed contracts	Intended to execute via delegatecall in a Safe context; direct calls revert because <code>address(this) == multisendSingleton</code> .



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
JuratSafe / JuratSafeL2	receive()	Native currency sent directly to the Safe proxy.	None	Any account or contract.	Funds become wallet assets controlled by owner execution, module execution, and recovery execution paths.
JuratSafe / JuratSafeL2	Direct token transfer to proxy address	ERC20/ ERC721/ ERC1155 or nonstandard tokens transferred by token contracts.	None during receipt.	Any token holder or token contract.	No Safe function runs for ordinary ERC20 receipt; accounting depends on token behavior and offchain indexing.
JuratSafe / JuratSafeL2	setup(...)	Optional initial funding can already exist at the proxy address before setup.	Optional setup payment through handlePayment if payment > 0.	Initial setup caller, intended factory deployment flow.	Setup permanently binds owners, modules, fallback handler, recovery emitter, and factory; setup payment can move assets before user operation begins.



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
JuratSafe / JuratSafeL2	execTrans actionFrom ModuleR eturnData(...)	Same as module execution.	Same as module execution.	Enabled module.	Return data can support integrations, but authorization remains module-only rather than owner- threshold.
RecoveryMa nager / JuratSafe	executeRe covery(Jur atRecover yCode)	None directly; recovery operates on assets already held by the Safe.	Native/ token assets can leave according to arbitrary callInfo[]; self-calls can mutate owners/ modules/ guards.	Current factory owner.	jrc.to is metadata only in current code; actual recipients/ selectors are determined by callInfo.
JuratSafe / JuratSafeL2	Owner- managem ent self- calls (addOwner WithThres hold, removeOw ner, swapOwne r, changeThr eshold)	None.	No direct asset transfer.	Safe self- call through owner transacti on, module, or recovery.	Changes who can later move assets; recovery self-call PoC showed owner/ threshold mutation without owner signatures.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
JuratSafe / JuratSafeL2	Module/fallback/guard self-calls (enableModule, disableModule, setFallbackHandler, setGuard)	None.	No direct asset transfer.	Safe self-call through owner transaction, module, or recovery.	Changes privileged execution and liveness surfaces that determine future asset movement.
MultiSendFees	multiSend(bytes)	Optional native currency in Safe context; arbitrary batched calls can also receive assets.	Batched call/delegatecall transfers plus factory-owner ETH fee from caller context.	Safe transaction via delegate call.	Batch calldata can hide multiple asset movements; fee path may create policy mismatch or double-charge risk if combined with direct execution fee.
SafeProxyFactoryWithFees	createProxyWithNonce(...) / createChainSpecificProxyWithNonce(...) / createProxyWithCallback(...)	None normally in factory; deployed proxy may already receive assets or run setup payment logic.	None normally from factory.	Public.	Initializer determines permanent wallet control-plane and may trigger setup-time asset movement from the new proxy.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
SafeProxyFactoryWithFees	setFee(uint256)	None.	No direct asset movement.	Factory Owner.	Changes future ETH fee outflows from wallets that resolve live factory fee.
RecoveryEventEmitter	executedRecovery(JuratRecoveryCode)	None	None	Public.	No direct asset movement, but spoofed logs can trigger offchain automation if consumers do not authenticate provenance.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of Jurat. We performed our audit according to the procedure described above.

2 High and 1 Medium severity issues were found, Which the Jurat Team has Fixed and Acknowledged.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

June 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com